

第一题

(1) (8分)

设有两个复数 $c = a + bi$ 、 $c' = a' + b'i$ ，其中 a 、 b 、 a' 、 b' 均为实数。请用最多三次实数间乘法计算出 c 与 c' 的乘积。

(2) (8分)

设函数 $T(n) : \mathbb{N}^+ \rightarrow \mathbb{R}^+$ 满足下列递推式，请简要证明 $T(n) = O(n^3 \log n)$ 。

$$\begin{cases} T(n) = \frac{64}{9}T\left(\lceil \frac{n}{2} \rceil\right) + 3T\left(\lceil \frac{n}{3} \rceil\right) + O(n^3) \\ T(1) = 1 \end{cases}$$

第二题 (14分)

设有一个长度为 n 的 p -进制数组（即每一位取值为 $0, 1, \dots, p-1$ ），用于实现一个无符号的 p -进制计数器（ $p \geq 2$ 为整数）。计数器初始值为 0。一次操作对计数器加 1：从最低位开始，若该位的值小于 $p-1$ ，则将当前位加 1 并停止；若该位的值等于 $p-1$ ，则将当前位设为 0 并向高一位进位，依此类推，直到不再需要进位或达到最高位（产生溢出）。记一次操作的代价为被修改的位的个数。请证明：计数器从 0 开始， k 次操作的总代价不超过 $\frac{kp}{p-1}$ 。方便起见，假设过程中没有产生溢出。

第三题

给定一个包含 n 个不同整数的数组 $A[1 \dots n]$ 和值 $0 < \epsilon \leq \frac{1}{2}$ 。设 B 为 A 的一个重排（permutation）。如果对每个 $i \in [1, n]$ 均有 $|i - \text{rank}(B[i])| \leq \epsilon n$ ，则称 B 为 A 的“ ϵ -近似排序”。此处， $\text{rank}(B[i])$ 为 $B[i]$ 在数组 A 中的排序序数（order statistic）。

(1) (10分)

请设计一个算法，对任意 A 、 ϵ ，其能在 $O(n \log(\frac{1}{\epsilon}))$ 的时间内计算一个 ϵ -近似排序 B 。要求简述算法思路，用自然语言描述算法过程。描述过程中，可以借助伪代码以方便表述。方便起见，假设 ϵn 为整数。（提示：找到 A 中所有序数为 ϵn 的整数倍的数会有所帮助。）

(2) (6分)

简要证明你的算法的正确性。

(3)(4分)

分析你的算法的时间复杂度。

第四题 (15分)

考虑一个包含 n 个不同整数的数组 A ，它是由一个原始升序数组经过一次循环右移得到的。也即，对某个整数 $k \in [1, n-1]$ ，原始升序数组最右侧的 k 个数字被整体移动到了数组的最

左侧。例如，原始数组是 $\{5, 15, 27, 29, 35, 42\}$ 时，如果 $k = 2$ ，则所得的 A 为 $\{35, 42, 5, 15, 27, 29\}$ 。假设你并不知道 k 的值，请设计一个时间复杂度为 $O(\log n)$ 的算法计算出 k 的值。要求简述算法思路，给出伪代码，并分析算法的时间复杂度。

第五题 (15分)

在一个包含 n 个整数的数组 $A[1 \cdots n]$ 中，如果任何一个长度为 k 的子数组都不包含相同元素——也即不存在 $1 \leq i < j \leq n$ 使 $j - i \leq k - 1$ 和 $A[i] = A[j]$ 同时满足，则我们称 A 是“ k -unique”的。此处， $k \geq 2$ 。请设计一个算法，能够在 $O(n \log k)$ 时间内判断 A 是否是“ k -unique”的。要求简述算法思路，用自然语言描述算法过程，并分析算法的时间复杂度。描述过程中，可以借助伪代码以方便表述。

第六题

你正在和 n 个朋友举办派对，每位朋友要么扮演平民，要么扮演狼人。你不知道谁是平民谁是狼人，但你的所有朋友互相之间都知道真实身份。已知平民的人数严格多于狼人的人数。你可以执行的“查询”操作如下：任选两位朋友，分别询问他们对方是平民还是狼人。在回答时，平民必须如实说出对方的身份，而狼人则不必（即狼人可能会对另一人的身份撒谎）。无论狼人表现出何种行为，你的算法都必须保证正确。

(1) (6分)

给定一名朋友作为输入，设计一个算法使用 $O(n)$ 次查询来确定该朋友是否为平民。要求简述算法思路，用自然语言描述算法过程，并简要说明算法正确性。描述过程中，可以借助伪代码以方便表述。

(2) (8分)

设计一个分治算法，使用 $O(n \log n)$ 次查询来找出一名平民。要求简述算法思路，用自然语言描述算法过程，并分析算法的时间复杂度。描述过程中，可以借助伪代码以方便表述。（提示：将人群分为两组，这两组中至少有一组必然满足什么不变量？）

- 必然有一组当中是平民大于狼人
- 一直划分，直到一个人

(3) (6分)

证明上述第 (2) 问找出平民的算法的正确性

第一题

(1)

设复数 $c = a + bi$ ， $c' = a' + b'i$ ，乘积为

$$c \cdot c' = (aa' - bb') + i(ab' + a'b).$$

通常需要四次实数乘法： aa' 、 bb' 、 ab' 、 $a'b$ 。

利用恒等式

$$(a+b)(a'+b') = aa' + ab' + a'b + bb',$$

可以只用三次乘法：

- 计算 $P_1 = aa'$,
 - 计算 $P_2 = bb'$,
 - 计算 $P_3 = (a+b)(a'+b')$ 。
- 则实部为 $P_1 - P_2$ ，虚部为 $P_3 - P_1 - P_2$ 。
- 因此最多三次实数乘法可得到复数乘积。

(2)

已知递推式

$$T(n) = \frac{64}{9}T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 3T\left(\left\lceil \frac{n}{3} \right\rceil\right) + O(n^3), \quad T(1) = 1.$$

我们证明 $T(n) = O(n^3 \log n)$ 。

使用代入法：假设存在常数 c, d 使得 $T(n) \leq cn^3 \log n + dn^3$ 对足够大的 n 成立（忽略取整影响）。代入右端：

$$\begin{aligned} T(n) &\leq \frac{64}{9}c\left(\frac{n}{2}\right)^3 \log \frac{n}{2} + 3c\left(\frac{n}{3}\right)^3 \log \frac{n}{3} + O(n^3) \\ &= \frac{64}{9} \cdot \frac{cn^3}{8} (\log n - 1) + 3 \cdot \frac{cn^3}{27} (\log n - \log 3) + O(n^3) \\ &= cn^3 \left[\frac{8}{9} (\log n - 1) + \frac{1}{9} (\log n - \log 3) \right] + O(n^3) \\ &= cn^3 \log n - cn^3 \cdot \frac{8 + \log 3}{9} + O(n^3). \end{aligned}$$

选取足够大的 c 使得 $c \cdot \frac{8 + \log 3}{9} \geq$ 隐含在 $O(n^3)$ 中的常数，则上式 $\leq cn^3 \log n$ （加上低阶项可被吸收）。因此 $T(n) = O(n^3 \log n)$ 。

第二题 (14分)

设 p -进制计数器，初始为 0。第 i 位（个位为第 0 位）每 p^i 次操作改变一次（因为从 0 到 $p-1$ 循环，每次加 1 时该位是否变化取决于低位的进位）。实际上，个位每次操作都改变，改变次数为 k ；十位每 p 次操作改变一次，次数为 $\lfloor k/p \rfloor$ ；百位为 $\lfloor k/p^2 \rfloor$ ，依此类推。

一次操作的代价是被修改的位数，即所有位改变次数之和。因此 k 次操作的总代价为

$$\sum_{i=0}^{\infty} \left\lfloor \frac{k}{p^i} \right\rfloor \leq \sum_{i=0}^{\infty} \frac{k}{p^i} = \frac{k}{1 - 1/p} = \frac{kp}{p-1}.$$

故总代价不超过 $\frac{kp}{p-1}$ 。

第三题

(1) (10分)

算法思路：

将数组 A 中元素按排名分成若干段，每段长度 ϵn （最后一段可能不足）。通过找出所有排名为 $\epsilon n, 2\epsilon n, \dots$ 的元素作为分界点，将元素分配到对应段中，再将这些段整体放置到相同排名区间的位置上。

算法步骤：

1. 令 $m = 1/\epsilon$ （由假设 ϵn 为整数知 m 为整数）。
2. 使用递归选择算法（如基于快速选择的二分法）在 $O(n \log m)$ 时间内找出 A 中所有秩为 $\epsilon n, 2\epsilon n, \dots, (m-1)\epsilon n$ 的元素，得到分界值序列 $p_1 < p_2 < \dots < p_{m-1}$ 。再添加 $p_0 = -\infty$ ， $p_m = +\infty$ 。
3. 创建 m 个空桶 $B_0, B_1, \dots, B_{m-1}$ 。遍历 A 中每个元素 x ，通过二分查找确定 j 使得 $p_j < x \leq p_{j+1}$ ($0 \leq j < m$)，将 x 放入桶 B_j 。
4. 构造结果数组 B ：初始化位置指针 $pos = 0$ 。对于 $j = 0$ 到 $m-1$ ，将桶 B_j 中的元素依次放入 $B[pos], B[pos+1], \dots$ ，然后将 pos 更新为 $(j+1)\epsilon n$ （因为每个桶应占用的位置区间长度为 ϵn ，最后一个桶可能不足，但位置区间相应缩小）。

伪代码：

```
function approximate_sort(A, ε):
    n = len(A)
    m = 1/ε
    pivots = find_quantiles(A, [εn, 2εn, ..., (m-1)εn]) // 返回有序分位数
    buckets = [[] for _ in range(m)]
    for x in A:
        j = binary_search(pivots, x) // 找到x所属区间索引
        buckets[j].append(x)
    B = [0]*n
    start = 0
    for j in range(m):
        for x in buckets[j]:
            B[start] = x
            start += 1
        start = (j+1)*εn // 跳过到下一个区间起点
    return B
```

(2) (6分)

正确性证明：

对任意元素 $x \in A$ ，设其排名为 r (1-indexed)。由构造知 x 被放入桶 B_j ，其中 $j = \lfloor r/(\epsilon n) \rfloor$ （边界情况类似）。因此 $j\epsilon n \leq r < (j+1)\epsilon n$ 。在数组 B 中，所有桶 B_j 的元素被放置在位置区间 $[j\epsilon n, (j+1)\epsilon n)$ 内（最后一个区间可能右端点小于 n ，但差值仍小于 ϵn ）。于是 x 的新位置 i 满足 $j\epsilon n \leq i < (j+1)\epsilon n$ 。从而 $|i - r| < \epsilon n$ 。故 B 是 ϵ -近似排序。

(3) (4分)

时间复杂度分析：

- 找分位数：递归选择算法每次将问题规模减半，总复杂度 $O(n \log m) = O(n \log(1/\epsilon))$ 。
 - 二分查找每个元素： $O(\log m)$ ，共 n 个元素，耗时 $O(n \log(1/\epsilon))$ 。
 - 填充桶： $O(n)$ 。
- 总时间复杂度 $O(n \log(1/\epsilon))$ 。

第四题 (15分)

算法思路：

数组由升序数组循环右移得到，其最小值位于旋转点之后（即原数组的第一个元素）。利用二分查找寻找最小值的位置，从而得到 k 。

伪代码：

```
function find_k(A):
    n = length(A)
    left = 0, right = n-1
    while left < right:
        mid = (left + right) // 2
        if A[mid] < A[right]:
            right = mid
        else:
            left = mid + 1
    // 此时 left 为最小值下标 (0-index)
    return left    // 因为最小值在原数组中下标0，右移后下标为k
```

时间复杂度：每次循环将区间减半，故 $O(\log n)$ 。

第五题 (15分)

算法思路：

使用滑动窗口和平衡二叉搜索树（如红黑树）维护当前窗口内的元素。窗口大小固定为 k ，每次右移时添加新元素、删除最左元素。若添加时发现元素已存在，则存在长度为 k 的子数组含有重复元素，返回 false；否则遍历结束返回 true。

算法过程：

1. 初始化一个空集合 S （支持 $O(\log k)$ 的插入、删除、查找）。
2. 初始化左指针 $l = 1$ 。
3. 对于右指针 $r = 1$ 到 n ：
 - 若 $A[r]$ 已在 S 中，返回 false。
 - 将 $A[r]$ 插入 S 。
 - 若 $r - l + 1 > k$ ，则从 S 中删除 $A[l]$ ，并将 l 加 1。
4. 返回 true。

时间复杂度：每个元素至多插入和删除一次，每次操作 $O(\log k)$ ，总 $O(n \log k)$ 。

第六题

(1) (6分)

算法：给定朋友 x ，对每个其他朋友 y 执行一次查询：询问 y “ x 是平民还是狼人？” 统计回答“平民”的次数 cnt 。若 $cnt > (n - 1)/2$ ，则 x 是平民；否则是狼人。

正确性：已知平民人数 $H > W$ （狼人数），且 $H + W = n$ 。

- 若 x 是平民，则所有平民（ H 人）会如实回答“平民”，而狼人可能回答任意，但至少 H 个回答为“平民”，且 $H > n/2$ ，所以 $cnt > (n - 1)/2$ 。
- 若 x 是狼人，则所有平民（ H 人）会回答“狼人”，故回答“平民”的人数最多为 $W < n/2$ ，所以 $cnt \leq (n - 1)/2$ 。

因此根据多数即可判断。查询次数 $n - 1 = O(n)$ 。

(2) (8分)

分治算法：

重复以下过程直到只剩一人：

1. 将当前人群两两配对，若人数为奇数则多出一人。
2. 对每一对 (a, b) ，询问 a 关于 b 的身份和 b 关于 a 的身份。
 - 若两人都回答“平民”，则保留 a （或任意一人）进入下一轮。
 - 否则，两人都淘汰。
3. 将未配对的人（若有）也保留进入下一轮。

最终剩下的一人即为平民。

伪代码：

```
function find_civilian(people):
    if |people| == 1:
        return people[0]
    next_round = []
    i = 0
    while i+1 < len(people):
        a, b = people[i], people[i+1]
        ans_a = query(a, b)    // a 说 b 的身份
        ans_b = query(b, a)    // b 说 a 的身份
        if ans_a == '平民' and ans_b == '平民':
            next_round.append(a)
        i += 2
    if i < len(people):        // 奇数个，落单
        next_round.append(people[i])
    return find_civilian(next_round)
```

时间复杂度：每轮查询次数不超过当前人数的一半，人数至少减半，故总查询次数 $\leq n/2 + n/4 + \dots = O(n) \leq O(n \log n)$ 。递归深度 $O(\log n)$ ，故算法复杂度 $O(n \log n)$ 。

(3) (6分)

正确性证明：用归纳法。

当人数为 1 时，由平民人数多于狼人，此人必为平民。

假设对人数小于 n 且平民多于狼人的集合，算法能正确返回平民。考虑人数为 n 的情形。设平民数 H ，狼人数 W ， $H > W$ 。将人群配对（可能有一人落单），考虑三类对：

- 两个平民：互答“平民”，保留一人，损失一个平民。
- 两个狼人：狼人可撒谎互答“平民”，从而保留一个狼人，损失一个狼人。
- 一平民一狼人：平民说狼人是狼人，狼人无论答什么，均不会出现互答“平民”，故两人都淘汰，损失一平民一狼人。

设 x 为两平民对数， y 为两狼人对数， z 为一平民一狼人对数，落单者身份为 s （平民或狼人）。则

$$H = 2x + z + h_s, \quad W = 2y + z + w_s,$$

其中 $h_s + w_s = 1$ 若落单，否则为 0。由 $H > W$ 得 $2x + h_s > 2y + w_s$ 。

下一轮保留的人数为 $x + y + (\text{落单与否})$ ，其中平民数为 $x + h_s$ ，狼人数为 $y + w_s$ 。由上述不等式知 $x + h_s > y + w_s$ ，即平民仍多于狼人。因此归纳假设成立，最终剩下的一人为平民。